

COMP2048: Theory of Computation

Dr. Kasra Khosoussi

Week 5

26/3/2026

Recap

So far, we have studied several models of computation:

- Finite Automata (DFAs/NFAs)
- Pushdown Automata (PDAs)

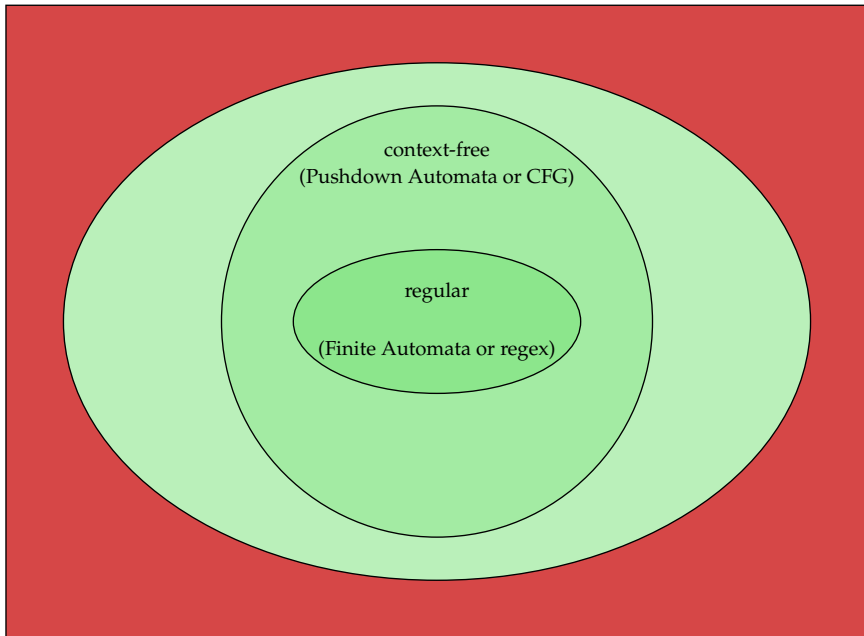
These models can recognize many languages, but:

- DFAs recognize only **regular languages**
- PDAs recognize **context-free languages**

There are still many problems they **cannot solve**.

- $L_1 = \{1^n 0^n \mid n \geq 0\}$ (not regular, but context-free)
- $L_2 = \{a^n b^n c^n \mid n \geq 0\}$ (not context-free)

Our Big Picture Revisited



Rectangle: set of all decision problems (or, languages) over Σ

What is missing?

Previous models have severe limitations:

- **Finite Automata:**

- No memory beyond the state

- **Pushdown Automata:**

- Only an unbounded **stack**
- Can only access the **top** of the stack

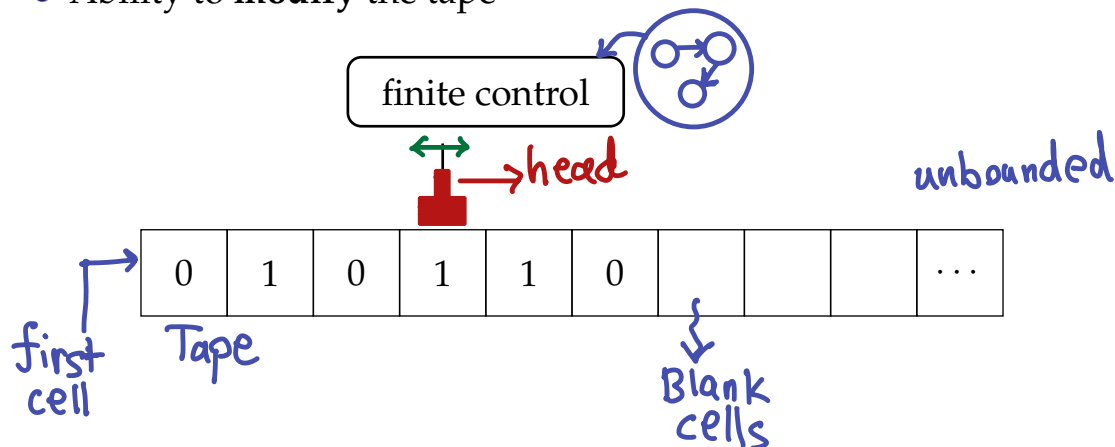
- Unbounded memory

- Ability to **read & write anywhere** (i.e., go back and forth)

Turing Machines

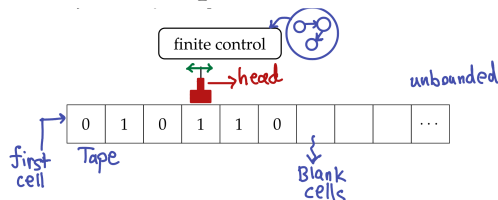
A New Model of Computation

- A finite state machine (a.k.a. finite control)
- An **infinite** tape (i.e., unbounded memory)
- A **read/write** head
- Ability to move left and right on the tape
- Ability to **modify** the tape



The Tape and Head

- One-sided infinite (to the right) tape divided into cells (i.e., as much **memory** as needed)
- Each cell contains a symbol from the **tape alphabet** Γ (e.g., $\Gamma = \Sigma \cup \{\sqcup\}$) ~~$\sqcup$~~ *blank*
- Head reads the symbol written on the current cell.
- Head will move *left* (L) or *right* (R) after reading a symbol.
- Initially:
 - Input string is placed on the tape.
 - The rest of the tape is blank (i.e., $\sqcup$).
 - Head starts on the leftmost symbol of the input.



Execution: Read-Write-State Transition-Head Move

- ① Read the current symbol under head.
- ② Apply transition function δ :
 - Write a (possibly new) symbol (from Γ) on the tape.
 - Move to a new state.
 - Move the head *left* or *right*.
- ③ Repeat 1 and 2 until:
 - The machine enters q_{accept} (**immediately halt** and accept).
 - The machine enters q_{reject} (**immediately halt** and reject).
 - The machine never halts (infinite loop – also called “reject by looping”).

Unlike previous models, TMs can accept/reject the input by entering the accept/reject states at **any time** (i.e., not exactly after reading the last symbol)

Example

`https://turingmachine.io`

Components of a Turing Machine

A Turing Machine M is formally a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

$$A \setminus B = A - B$$

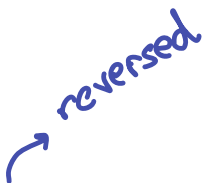
↓
set

where:

- Q : finite set of states.
- Σ : input alphabet (does not include the blank symbol).
- Γ : tape alphabet
(includes Σ and the blank symbol $\sqcup$ and possibly more).
- $\delta : Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\} \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$: transition function.
- $q_0 \in Q$: start state.
- $q_{\text{accept}} \in Q$: accept state.
- $q_{\text{reject}} \in Q$: reject state ($q_{\text{reject}} \neq q_{\text{accept}}$).

$$\text{e.g., } \delta(q, a) = (q', b, R)$$

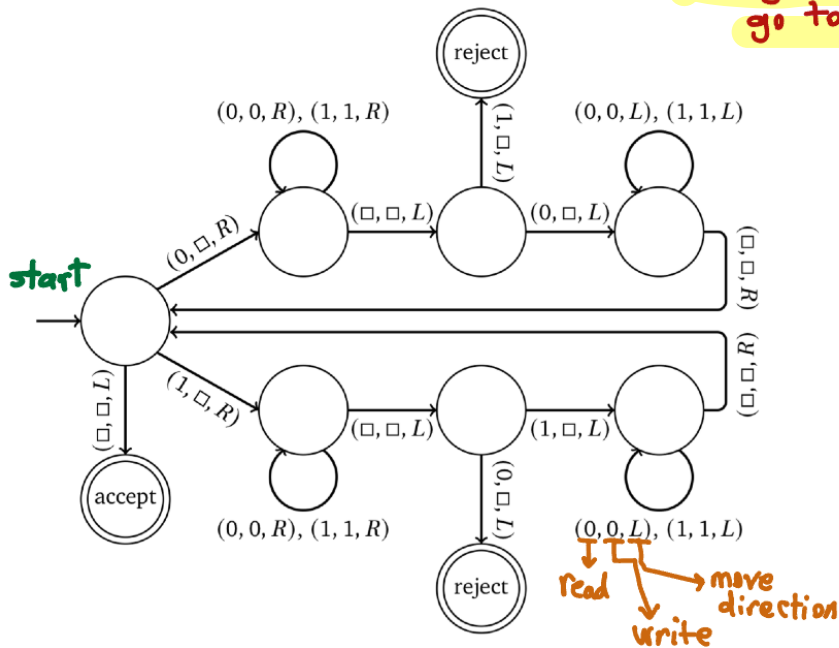
Example: Even-length Palindromes


$$L = \{ww^R \mid w \in \{0,1\}^*\}$$

Examples:

- $1001 \in L$
- $1000 \notin L$
- $10100101 \in L$

missing transitions
go to reject



credit: Dr. Richard Kueng

input: $w = 100$
 head
 blank



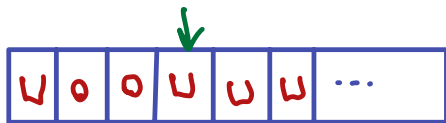
at q_0



at q_4



at q_4



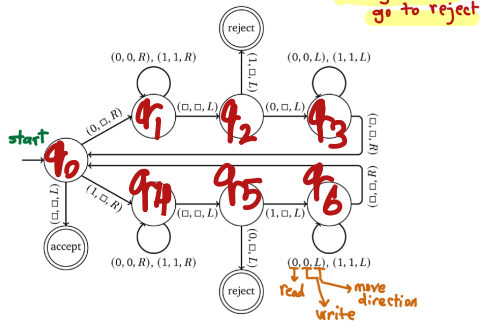
at q_4



q_5



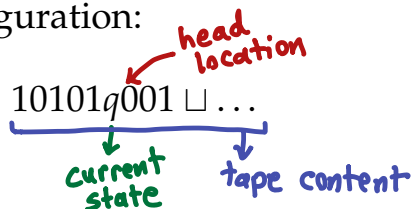
q_{reject}



credit: Dr. Richard Kueng

Configurations

- A **configuration** of a TM is a snapshot of the machine at any point
- Specifies:
 - 1 Current tape contents.
 - 2 Current head position.
 - 3 Current state $q \in Q$.
- **Initial configuration:**
 - input on the tape followed by blanks
 - head at the start of input
 - machine in the initial state q_0
- Useful notation for configuration:



Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

001

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

1. **Locate the leftmost unmarked symbol.**

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

❶ **Locate the leftmost unmarked symbol.**

- If none found (all are marked), *accept*.

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

❶ **Locate the leftmost unmarked symbol.**

- If none found (all are marked), *accept*.
- If it is a 1, *reject*.

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

① **Locate the leftmost unmarked symbol.**

- If none found (all are marked), *accept*.
- If it is a 1, *reject*.
- Otherwise, it is a 0. Mark it (e.g., replace with X).

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

- ❶ **Locate the leftmost unmarked symbol.**
 - If none found (all are marked), *accept*.
 - If it is a 1, *reject*.
 - Otherwise, it is a 0. Mark it (e.g., replace with X).
- ❷ **From there, scan right to find the rightmost unmarked symbol.**

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

- ① **Locate the leftmost unmarked symbol.**
 - If none found (all are marked), *accept*.
 - If it is a 1, *reject*.
 - Otherwise, it is a 0. Mark it (e.g., replace with X).
- ② **From there, scan right to find the rightmost unmarked symbol.**
 - If it is a 0, *reject* (we needed a 1).

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

① **Locate the leftmost unmarked symbol.**

- If none found (all are marked), *accept*.
- If it is a 1, *reject*.
- Otherwise, it is a 0. Mark it (e.g., replace with X).

② **From there, scan right to find the rightmost unmarked symbol.**

- If it is a 0, *reject* (we needed a 1).
- If no unmarked 1 is found, *reject*.

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

① Locate the leftmost unmarked symbol.

- If none found (all are marked), *accept*.
- If it is a 1, *reject*.
- Otherwise, it is a 0. Mark it (e.g., replace with X).

② From there, scan right to find the rightmost unmarked symbol.

- If it is a 0, *reject* (we needed a 1).
- If no unmarked 1 is found, *reject*.
- Otherwise, mark the 1 (e.g., replace with Y).

Design a TM to check if $w \in \{0^n 1^n \mid n \geq 0\}$

Idea: Pair each leftmost unmarked symbol with the rightmost unmarked symbol and mark them.

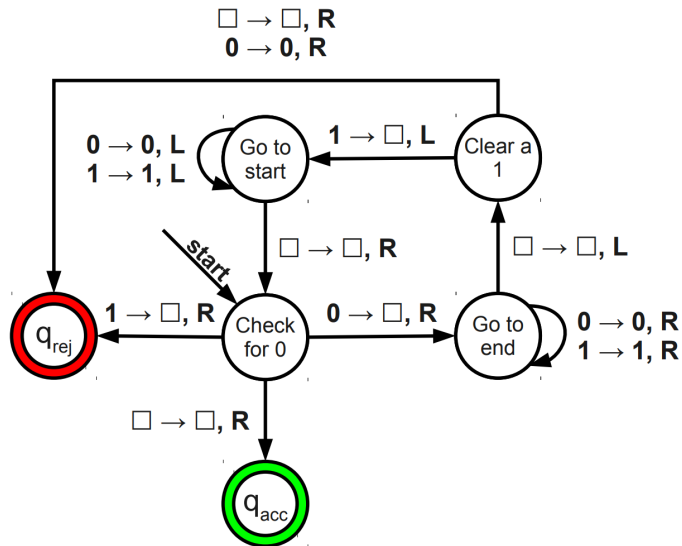
1 Locate the leftmost unmarked symbol.

- If none found (all are marked), *accept*.
- If it is a 1, *reject*.
- Otherwise, it is a 0. Mark it (e.g., replace with X).

2 From there, scan right to find the rightmost unmarked symbol.

- If it is a 0, *reject* (we needed a 1).
- If no unmarked 1 is found, *reject*.
- Otherwise, mark the 1 (e.g., replace with Y).

3 Repeat from Step 1 until all symbols are matched or a mismatch is discovered.



Courtesy of Sean Szumlanski

Common TM Design Patterns



- **Marking** symbols (e.g., replace with X/Y)
- **Scanning** left/right
- **Returning** to the start
- **Matching pairs**
- **Erasing / rewriting**
- **Use subroutines** (i.e., a group of states performing a task)

Remarks:

- You *can* overwrite each matched 0 or 1 with a blank to “erase” it.
- Make sure your TM doesn't confuse these newly written blanks with the *originally empty* cells beyond the input.
- Can always add additional symbols to the tape alphabet Γ to avoid confusion (e.g, boundary markers, special symbols for “erased 0”, “erased 1”, etc)
- Usually it's a good idea to initially verify if the input is in a valid form (e.g., by checking a regex – no need to write on the tape)

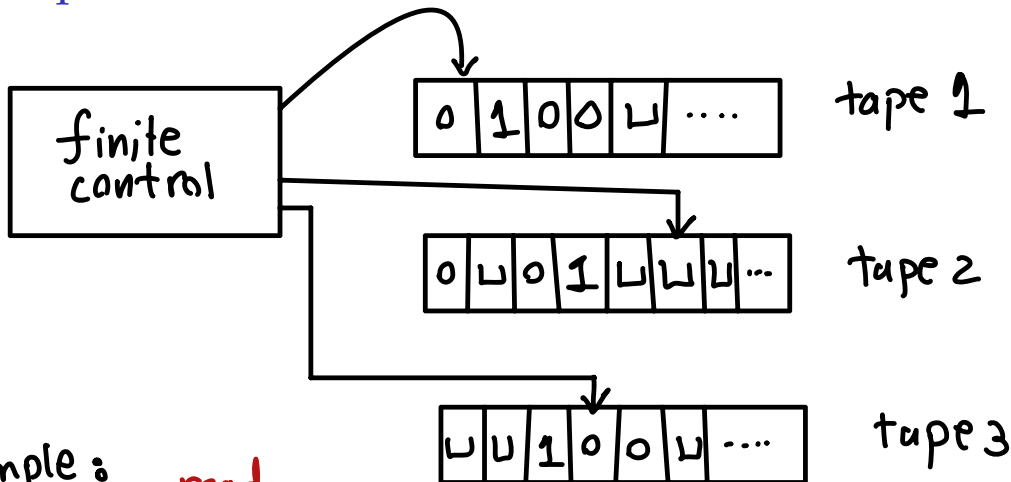
Zoo of TM Variants



- **Two-way Infinite Tape TM**
 - **Multi-tape TM:**
Multiple (finitely many) tapes, each with its own head.
 - **Non-deterministic TM:**
 - Transition function can branch to multiple possibilities.
 - Accept if *at least one* branch accepts (similar to NFA)
- many more ...

All *equivalent* in power to the standard single-tape deterministic TM!

Multi-tape TM



example :

$$\delta(q, \overbrace{a_1, a_2, a_3}^{\text{read}}) = (q', \underbrace{b_1, b_2, b_3}_{\text{write}}, \underbrace{L, L, R}_{\text{move}})$$

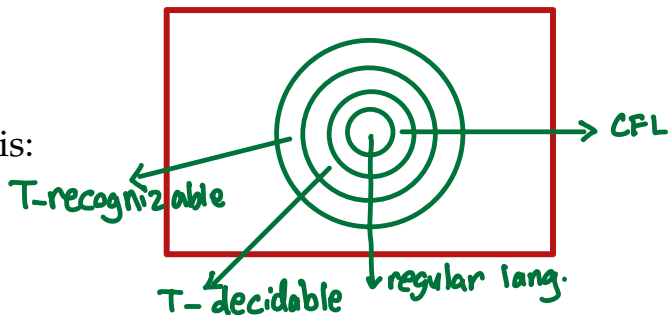
current state

next state



Definitions

We say a language $L \subseteq \Sigma^*$ is:



- **Turing-Recognizable:**

If there exists a TM that *halts and accepts* for $w \in L$ and either rejects or loops if $w \notin L$.

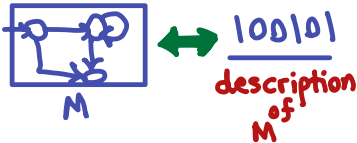
- **Turing-Decidable:**

If there exists a TM that:

- 1 halts on all inputs
- 2 accepts all $w \in L$ and rejects all $w \notin L$.

Regular lang. $\subset$ CFL $\subset$ T-decidable $\subset$ T-recognizable
↓
strict subset of

Universal Turing Machine (UTM)

- Recall that we can encode virtually everything as strings:
e.g., images [JPG], audio [MP3], text [e.g., ASCII or Unicode], etc
- We can even encode (blueprint of) **Turing Machines!**
(no fundamental distinction between code and data)
- Encode Turing machine M as $\langle M \rangle \in \Sigma^*$

- Intuitively, you can think of it as the “source code” (instructions, blueprint, etc) of the TM M

Universal Turing Machine Cont'd.

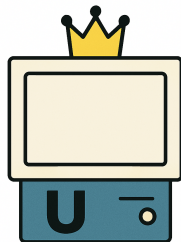
- There exists a **TM** (called the *Universal TM*) U that takes as input:
 - ① (encoding of) **any TM** M , and
 - ② **any input string** $w \in \Sigma^*$

Universal Turing Machine Cont'd.

- There exists a **TM** (called the *Universal TM*) U that takes as input:
 - ① (encoding of) **any TM** M , and
 - ② **any input string** $w \in \Sigma^*$and **simulates** M on w

Universal Turing Machine Cont'd.

- There exists a **TM** (called the *Universal TM*) U that takes as input:
 - ① (encoding of) **any TM** M , and
 - ② **any input string** $w \in \Sigma^*$and **simulates** M on w
 - if M accepts $w \Rightarrow U$ will accept $\langle M, w \rangle$
 - if M rejects $w \Rightarrow U$ will reject $\langle M, w \rangle$
 - if M loops on $w \Rightarrow U$ will loop on $\langle M, w \rangle$
- Treats $\langle M \rangle$ as a “program” and simulates M step by step on w .
- This concept underpins the idea of **stored-program computers**.
- Invention of **general-purpose computer!**



History

- **1900 Hilbert's Program**

- David Hilbert proposes 23 problems
- Goal: Formalize all of mathematics (complete, consistent, decidable)



- **1928 Entscheidungsproblem**

- Hilbert & Ackermann pose the decision problem
- Is there an “**algorithm**” to decide truth of any statement?

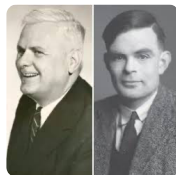


- **1931 Gödel**

- Incompleteness Theorems
- Any consistent formal system strong enough for arithmetic is incomplete

- **1936 Church & Turing**

- Church: Undecidability via lambda calculus
- Turing: Turing machines & Halting Problem
- They essentially defined what an “algorithm” is (e.g., a TM)
- Result: Entscheidungsproblem is unsolvable



Church-Turing Thesis



Church-Turing Thesis

Any function that can be computed by an “algorithm” can be computed by a Turing Machine.

- Not a *formal theorem*, but a widely accepted belief.
- No counterexamples so far.
- Equivalences:
 - λ -Calculus
 - Other models of computation (e.g., by Post, etc)
- All major programming languages are **Turing-complete**



Accidentally Turing-Complete

Some things were not supposed to be Turing-complete. This is a collection of such accidents.

Stuff which is somehow limited (stack overflows, arbitrary configuration, etc) is still considered Turing complete, since all "physical" Turing machines are resource limited.

C++ Templates

Although they were initially not supposed to, C++ templates are Turing-complete. For proof look at this [paper \(pdf\)](#).

TypeScript Type System

In a very similar fashion to C++, the type system of TypeScript can be used to [implement a prime check](#).

Java Generics

Although Java set out to fix the errors of C++, it also fell into Turing-completeness eventually. [Radu Grigore built a parser generator for fluent interfaces](#).

X86 Mov

The paper [mov is Turing-complete](#) starts as this:

It is well-known that the x86 instruction set is baroque, overcomplicated, and redundantly redundant. We show just how much fluff it has by demonstrating that it remains Turing-complete when reduced to just one instruction.

It inspired [movfuscator](#), the single instruction C compiler. Building on this, there is a [branchless, mov-only version of the classic DOOM video game](#).

This is thought to be entirely secure against the Meltdown and Spectre CPU vulnerabilities, which require speculative execution on branch instructions.

The mov-only DOOM renders approximately one frame

Share page on

Twitter

Facebook

HTML5 + CSS3

While older HTML/CSS versions are not, with some new additions, you can write a [rule 110 automaton](#). The original page has moved here and there is also [Github](#) and a video.



Minecraft

Maybe this one was intentional, but the complexity of the computers people build in Minecraft is impressive.



Dwarf Fortress

In a similar vein, the infamous Dwarf Fortress game provides various ways to build logical AND, OR, or NAND gates for [computing](#). A very impressive example is a space invaders game in the following video.

